

Segmentação de Imagens por Análise de Textura

Maria C. Sauer¹

¹Departamento de Informática – Universidade Federal do Paraná (UFPR)

mcs22@inf.ufpr.br

1. Introdução

O trabalho tem por objetivo desenvolver um algoritmo de visão computacional para segmentar imagens por meio da análise de textura. O sistema foi implementado em Python com o auxílio da biblioteca OpenCV.

2. Pré-Processamento das Imagens

Antes de aplicar os filtros de textura, o conjunto de dados passou por uma etapa inicial de preparação para padronizar as entradas:

- **Recorte e Redimensionamento:** As imagens foram centralizadas e recortadas no formato quadrado e em seguida sofreram um redimensionamento para 512×512 pixels.
- **Conversão de Cor:** Todas as imagens RGB foram convertidas para escala de cinza.

3. Metodologia e Escolha de Parâmetros

Para construir os vetores que descrevem cada região da imagem em termos de textura, a técnica de extração por banco de filtros foi implementada. O pipeline foi feito seguindo as etapas:

3.1. Construção do Banco de Filtros

Foram criados 8 filtros no total: 4 filtros direcionais de Gabor para capturar as orientações horizontal (0 graus), vertical (90 graus), 45 e 135 graus, e 4 filtros circulares do tipo Laplaciano do Gaussiano.

Os filtros de Gabor atuam como detectores direcionais, responsáveis por capturar linhas em angulações específicas. Já o Laplaciano do Gaussiano tem o papel de filtro circular, fazendo detecção de padrões não direcionais, como pontos ou manchas espalhadas pela imagem. Juntos, eles são capazes de detectar bordas, barras e pontos em múltiplas orientações.

Na configuração dos filtros de Gabor, foi definido um tamanho de kernel igual a 21 e um lambda de 8. Esses valores garantem que o padrão visual procurado se encaixe dentro do espaço de análise. Outros valores foram testados (como o kernel em 15 e em 7 e o lambda em 5, 7 e 9), no entanto, a combinação escolhida foi a que apresentou o melhor resultado para o dataset selecionado.

3.2. Processamento Multiescala

As imagens originais sofreram reduções pela metade de forma sucessiva, por meio da aplicação de um filtro gaussiano, totalizando o processamento em 3 escalas diferentes. Para unir os dados, foi calculada a média das respostas de textura utilizando uma janela de blur de 7x7 (da mesma forma que com o banco de filtros, outros valores foram testados, apesar de 15x15 apresentar um desempenho mediano, a imagem do animal se misturava ainda mais com o fundo da imagem). A união dos resultados nas três escalas gerou um vetor com 24 dimensões.

3.3. Agrupamento por Similaridade

A segmentação foi feita utilizando a distância euclidiana N-dimensional para encontrar regiões com texturas parecidas. O agrupamento foi feito usando K-means, que mapeou os pixels em diferentes cores com base em seus vetores de 24 dimensões.

4. Discussão

A análise dos resultados mostrou aspectos importantes do algoritmo:

- **Carga de Informação Visual:** Em imagens com excesso de elementos, as janelas capturam informações variadas. Resultando em vetores complexos que dificultam a separação precisa entre o objeto e o fundo da imagem.
- **Importância do Contraste:** O algoritmo apresenta resultados mais definidos em cenas com maior contraste. Técnicas de pré-processamento auxiliariam na melhoria da visibilidade das texturas, melhorando os resultados.
- **Variação de Escala e Distância:** Como o sistema lida com variações de escala, elementos fotografados muito de perto ou de longe perdem precisão na segmentação, uma vez que a repetição do padrão foge da faixa escolhida para os filtros fixos (ksize e lambda).

5. Resultado

Para ilustrar o funcionamento da extração de características, a Figura 1 e a Figura 2 apresentam, respectivamente, a imagem original em tons de cinza e o seu agrupamento final.

Aqui vemos um caso de sucesso do algoritmo. Percebemos que existe um alto contraste entre o animal e o fundo na Figura 1.



Figura 1. Imagem original em tons de cinza.

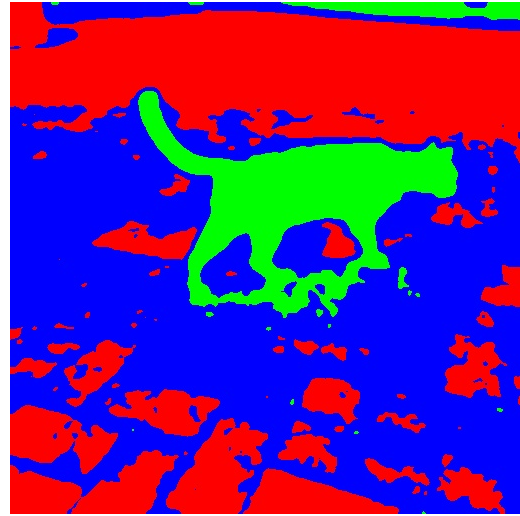


Figura 2. Segmentação após agrupamento K-Means.

Por outro lado, aqui vemos um caso de falha do algoritmo. Percebemos que existe um baixo contraste entre o animal e o fundo na Figura 3.



Figura 3. Imagem original em tons de cinza.

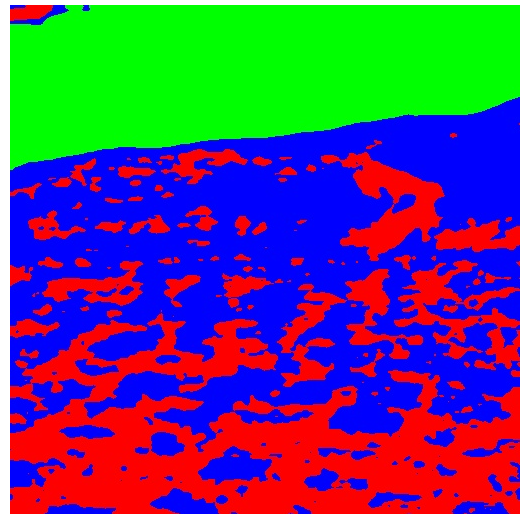


Figura 4. Segmentação após agrupamento K-Means.

6. Código-Fonte

A implementação e o conjunto de imagens analisadas estão no GitHub:

https://github.com/mariasauer/visao_computacional